

Universidad Tecnológica de Bolívar

Informe investigativo – Docker

Presentado por:

Ana María Rojas Mendoza

Código estudiantil:

T00082483

Curso: Arquitectura de Software

NRC: 1508

Docente: Luis Giovanny Carreno Ortiz

Cartagena, Bolívar

Octubre de 2026

Introducción

En la actualidad, el momento de desarrollar un programa puede requerir que este funcione en ordenadores diferentes o servidores. Esto puede ser un inconveniente ya que cada uno de ellos puede tener una configuración diferente y por este motivo a veces ocurren errores a la hora de ejecutar la aplicación. Existen herramientas que nos pueden ayudar como es el caso de Docker.

Docker es una herramienta que permite almacenar una aplicación junto con todo lo necesario para que dicha aplicación funcione. De esta forma, al ejecutar esto sobre diferentes máquinas no se tiene que volver a configurar todo de nuevo en cada lugar diferente. Por esta razón Docker se usa en el desarrollo de software hoy en día y mucho más cuando se desarrolla con microservicios y otros sistemas en los que una aplicación está dividida.

¿Qué es Docker?

Docker es una herramienta que permite crear, compartir y ejecutar aplicaciones utilizando contenedores. Esos contenedores incluyen todo lo que necesita un programa para funcionar: como su código, las bibliotecas, las configuraciones y otros elementos.

Docker crea espacios separados llamados contenedores, donde cada aplicación puede funcionar sin interferir con las demás. Aunque cada contenedor trabaja de forma independiente, todos comparten una parte del sistema operativo del computador donde se ejecutan. Esto hace que Docker sea más ligero, ya que no es necesario instalar un sistema operativo completo para cada aplicación.

A diferencia de una máquina virtual, un contenedor no necesita instalar un sistema operativo completo para cada aplicación. Las máquinas virtuales incluyen su propio sistema operativo. En cambio, los contenedores comparten el sistema operativo del host.

Componentes

Docker está compuesto por varios elementos que trabajan juntos para ejecutar aplicaciones:

- CLI (Docker Client): Es la herramienta con la que el usuario le da instrucciones a Docker mediante comandos. Por ejemplo, se utiliza para crear, iniciar o detener contenedores.
- Daemon (dockerd): Es el proceso que funciona en segundo plano y se encarga de realizar las tareas que le pide el usuario.
- Imagen: Es como una plantilla que contiene los archivos y elementos necesarios para ejecutar una aplicación. A partir de ella se pueden crear uno o varios contenedores.
- Contenedor: Es el entorno donde se ejecuta una aplicación a partir de una imagen.
- Dockerfile: Es un archivo de texto en el que se escriben las instrucciones que Docker debe seguir para crear una imagen.
- Registro: Es un lugar donde se guardan las imágenes de Docker para que otras personas puedan descargarlas o para que el usuario las comparta.

Dockerfile-(docker build)→ Imagen-(docker run) →Contenedor-(docker push / docker pull) → Registro (Docker Hub)

Funcionalidades principales

1. Portabilidad
Una imagen Docker puede ejecutarse en cualquier computador que tenga Docker instalado.
Ejemplo:
Un desarrollador crea una API en su computador Windows y otro miembro de su equipo puede ejecutarla exactamente igual en Linux utilizando la misma imagen.

2. Aislamiento
Cada contenedor tiene su propio entorno de ejecución.
Ejemplo:
Un contenedor puede utilizar MySQL 8 mientras otro utiliza PostgreSQL sin generar conflictos entre ambos.

3. Reproducibilidad del entorno
Todos los integrantes de un equipo trabajan bajo las mismas condiciones.
Ejemplo:
Si diez desarrolladores usan la misma imagen Docker, todos tendrán exactamente las mismas configuraciones.

4. Escalabilidad
Docker permite crear múltiples instancias del mismo servicio.
Ejemplo:
Una tienda en línea puede ejecutar tres contenedores de su API para atender una mayor cantidad de usuarios.

Comandos principales

a)

Comando	¿Qué hace?	Ejemplo
docker pull	Descarga una imagen desde un registro	docker pull mysql:8
docker run	Crea y ejecuta un contenedor	docker run nginx
docker ps	Lista contenedores activos	docker ps
docker ps -a	Lista todos los contenedores	docker ps -a
docker logs	Muestra registros de un contenedor	docker logs api
docker exec	Ejecuta comandos dentro de un contenedor	docker exec -it mysql bash
docker stop	Detiene un contenedor	docker stop api
docker start	Inicia un contenedor detenido	docker start api
docker rm	Elimina contenedores	docker rm api
docker images	Muestra imágenes disponibles	docker images
docker rmi	Elimina imágenes	docker rmi nginx
docker build	Construye una imagen	docker build -t mi-api .
docker compose up	Levanta todos los servicios definidos	docker compose up

b) docker run*:* Permite crear e iniciar un contenedor.
Parámetros comunes: -d, -p, -p 8080:80, -e

docker exec*:* Permite acceder o ejecutar comandos dentro de un contenedor activo.
Parámetros comunes: -it
docker compose up*:* Inicia todos los servicios definidos en un archivo Docker Compose.
Parámetros comunes: -d

¿Qué problema resuelve?

Docker surge para solucionar el problema "En mi máquina funciona". Este problema ocurre cuando una aplicación funciona correctamente en el computador del desarrollador, pero falla al ejecutarse en otro entorno. Las causas comunes pueden ser diferentes versiones de dependencias, sistemas operativos distintas configuraciones incompatibles, instalaciones manuales incompletas, bibliotecas faltantes.

Antes de Docker
Un desarrollador crea una aplicación con Node.js 18 y MySQL 8.
Al enviarla a otro miembro del equipo, este tiene Node.js 20 y MySQL 5.7. Como resultado, la aplicación genera errores y no funciona correctamente.

Después de Docker
El desarrollador crea una imagen Docker que incluye Node.js 18 y todas las dependencias necesarias.
El otro miembro del equipo simplemente ejecuta: docker compose up
y obtiene exactamente el mismo entorno de trabajo.

¿Por qué es importante dentro de una Arquitectura de Software?

Docker es una herramienta importante en la Arquitectura de Software porque permite que cada parte de un sistema, como una API o una base de datos, funcione de manera independiente en su propio contenedor. Esto facilita el mantenimiento y ayuda a evitar problemas entre las dependencias de cada servicio. Además, permite mantener la misma configuración durante el desarrollo, las pruebas y la ejecución de una aplicación, reduciendo posibles errores. También facilita el crecimiento de los sistemas y su traslado a la nube. Por estas razones, Docker se utiliza en arquitecturas modernas.

Conclusión

Docker ha cambiado la forma de desarrollar y ejecutar aplicaciones, ya que permite trabajar con contenedores que funcionan de manera similar en diferentes equipos, evitando muchos problemas de configuración.

En la Arquitectura de Software, facilita el trabajo con microservicios, bases de datos y otros componentes. Personalmente, lo utilizaría en proyectos de la universidad que incluyan una API y una base de datos, porque hace más sencilla la instalación y permite que todos los integrantes del equipo trabajen con la misma configuración.

Referencias

Docker Inc. (2026). *What is Docker?* Recuperado de: https://www.docker.com/resources/what-container/

Oracle. (2026). Docker Explained. Recuperado de: https://www.oracle.com/cloud/cloud-native/container-registry/what-is-docke

GeeksforGeeks. (2026). *Docker Architecture Explained*. Recuperado de: https://www.geeksforgeeks.org/docker-architecture/

Amazon Web Services (AWS). (2026). *What is Docker?* https://aws.amazon.com/docker/